

Technical Documentation Frameworks: Inclusive Language, Cognitive Accessibility, and Automated Enforcement at Scale

The evolution of technical documentation has transitioned from basic style harmonization to a strategic discipline focused on usability, accessibility, and global reach.¹ While foundational initiatives like inclusivenaming.org perform vital functions in deprecating deeply offensive technical terms, they operate on a narrow scope.³ Modern technical communication strategies recognize that true inclusivity cannot be achieved merely by replacing highly visible exclusionary terms; it requires a systematic overhaul of prose structure, cognitive accessibility, and operational methodologies.⁴

A comprehensive documentation framework must integrate open-licensed style guidelines, address the psychological barriers of diminishing language, and implement automated "docs-as-code" enforcement mechanisms.⁶ Failing to look beyond macro-level technical terms introduces significant gaps, resulting in high cognitive load, localized translation friction, and systematic exclusion of neurodivergent or distracted readers.⁸

Open-Licensed and Publicly Referenceable Style Guides

Rather than authoring guidelines from scratch, technical communications teams rely on established, publicly accessible, or openly licensed style guides.¹¹ These resources serve as upstream foundations that can be modified to suit specific organizational contexts.¹³

The Google Developer Documentation Style Guide

Licensed under the Creative Commons Attribution 3.0 (CC-BY 3.0) license, the Google Developer Documentation Style Guide provides an exhaustive framework for creating clear, consistent, and inclusive technical documentation.¹² The guide approaches inclusivity through the lens of precision and global clarity.¹⁵ It offers actionable strategies for addressing gendered, ableist, ageist, and culturally specific language.¹⁵

A key strength of the Google framework is its direct guidance on writing around legacy non-inclusive terms that cannot be immediately removed because they are deeply embedded in codebases.¹⁵ When encountering technical key-terms like *master* or *slave* in existing configurations, the guide advises authors to minimize their use by formatting them strictly in code font, introducing them parenthetically on first use, and utilizing inclusive alternatives—such as *parent node* or *replica*—in all subsequent conceptual text.¹⁵

Furthermore, the guide addresses accessibility barriers by restricting ableist language.¹⁵ It

advises replacing common idioms like "sanity check" with "validation check" or "final check," and "cripples the service" with "slows down the service".¹⁵ To prevent the alienation of disabled or neurodivergent readers, the guide explicitly forbids describing individuals without disabilities as "normal" or "healthy," recommending neutral, specific terminology such as "nondisabled," "sighted," "hearing," or "neurotypical".¹⁵

The Microsoft Writing Style Guide

The Microsoft Writing Style Guide is a standard reference for technical writers and is widely adopted as a secondary upstream resource by open organizations.¹³ Microsoft's guide is notable for its strict emphasis on simplicity and global communication.¹⁰ It instructs writers to avoid modifier stacks—long chains of modifying words that confuse both native and non-native English speakers—and to balance a friendly tone with precise, grammatically conventional English.¹⁰

Crucially, Microsoft provides a rigorous framework for eliminating unnecessary adverbs and qualifiers.¹⁷ The guide advises omitting words like *quite*, *very*, *quickly*, *easily*, and *effectively* unless they are essential to the meaning of a statement, warning authors that assuming a task is easy for the customer can alienate readers.¹⁷

Specialized and Community-Driven Guidelines

To complement enterprise-level style guides, organizations leverage specialized frameworks that address community-specific or highly technical contexts:

- **Mozilla Developer Network (MDN) Writing Style Guide:** Highly regarded in web development, the MDN guide structures content around the "Three Cs" of writing: clarity, conciseness, and consistency.¹⁹ It explicitly details the avoidance of exclusionary idioms (for example, replacing "kill two birds with one stone" with "solve two problems at once") and outlines strict rules for accessible headings and numerical formatting to ensure screen-reader compatibility.¹⁹
- **Academy Software Foundation (ASWF) and Alliance for OpenUSD (AOUSD) Inclusive Language Guide:** This collaborative guide addresses technical terminology in open-source codebases.³ It categorizes non-inclusive language into socially charged terms, ableist language, ageist language, and violent metaphors.³ It offers precise replacements for common industry metaphors, such as replacing "grandfathered" or "legacy" with "flagship" or "rollover," and "dummy" with "placeholder" or "stub".³
- **The 18F Content Guide and plainlanguage.gov:** Maintained by the United States government and licensed in the public domain, these guides focus on usability, plain language, and cognitive accessibility.²⁰ They are frequently used as core reference materials for building automated linting rules due to their emphasis on straightforward, structured content.²¹

The Cognitive Barriers of Diminishing Language

A significant gap in standard technical terminology audits is the failure to recognize

"diminishing language" as an accessibility barrier.⁷ Words such as *just, simply, obviously, clearly, basically,* and *easy* are often used in documentation in an attempt to make instructions appear approachable.²² In practice, this language introduces significant psychological and cognitive obstacles.¹⁸

The Psychology of the "Illusion of Ease"

In technical communication, the use of diminishing adverbs constructs an "illusion of ease".⁷ When documentation asserts that a step is "simple" or "obvious," it establishes an unwritten expectation of immediate comprehension.¹⁸ If a reader encounters difficulty—due to differences in technical background, environmental distractions, or cognitive processing—the language shifts the perceived failure onto the user.²⁴

This dynamic causes cognitive friction, increases anxiety, and often leads to task abandonment.²⁴ In professional settings, this translates directly to increased customer frustration and higher support ticket volumes.⁴

Cognitive Accessibility and WCAG Standards

Cognitive accessibility focuses on designing content that accommodates variations in human cognition, including learning disabilities, ADHD, dyslexia, traumatic brain injuries, and age-related cognitive decline.⁹ It is estimated that approximately 15% to 20% of the global population experiences some form of language processing challenge.⁹

The Web Content Accessibility Guidelines (WCAG 2.2) require digital content to be *Perceivable, Operable, Understandable,* and *Robust*.⁴ Diminishing language directly conflicts with the *Understandable* principle.⁴ WCAG Success Criterion 3.1.5 (Reading Level) emphasizes the necessity of plain language, concise sentences, and the elimination of unnecessary modifiers.⁴ The W3C Cognitive Accessibility Task Force (COGA) recommends the systematic removal of abstract metaphors, complex punctuation, and condescending phrasing.⁹ For example, the COGA task force notes that slash notations, such as "and/or," create significant reading hurdles for individuals with dyslexia or ADHD, as well as for users navigating with screen readers.²⁸

Deficit-Based versus Asset-Based Framing

In addition to removing diminishing adverbs, modern writing guidelines advocate for a shift from deficit-based to asset-based framing.²⁹ Deficit-focused language defines individuals or systems by their limitations or perceived lacks, whereas asset-based framing focuses on capability, agency, and neutral facts.²⁴

For example, using terms like "wheelchair-bound" or "suffering from a disability" portrays the individual as a passive victim.¹⁵ Inclusive frameworks require neutral, factual phrasing, such as "uses a wheelchair" or "experiencing a disability".¹⁵

This distinction also applies to paternalistic terminology.²⁹ The University of Illinois Chicago (UIC) Inclusive Language Guide recommends limiting the use of the word "empower," noting that the term implies an institution or author holds structural control and is granting permission

or power to an otherwise powerless recipient.²⁹ Instead, documentation should focus on providing tools and resources that allow readers to activate their own agency.²⁹

Overlooked Strategic Gaps in Documentation Audits

Organizations conducting inclusive language audits often miss systemic, structural issues that impact the scalability, translation, and overall quality of their documentation.¹ These gaps can be categorized into localization friction, literacy exclusion, and the financial impact of poor clarity.¹

Global Localization and Translation Friction

Technical documentation is routinely translated or localized for global markets.¹ Using metaphors, idiomatic expressions, or unnecessary modifiers significantly complicates the translation process.¹

Linguistic fillers like *actually*, *absolutely*, *could possibly*, or *assemble together* add unnecessary words that confuse machine translation tools and increase localization costs.⁷ Adhering to plain language principles directly reduces translation complexity, ensuring that the core technical instructions remain clear across multiple languages.¹

Literacy Exclusion

Writing for a global audience requires recognizing that a significant portion of readers may not use English as their primary language.⁸ The use of complex sentence structures, rare vocabulary, and culturally specific idioms results in "literacy exclusion".¹⁰

To prevent this, style guides like those from Microsoft and Mailchimp advocate for short, simple sentences that follow standard word order (subject + verb + object) and use consistent terminology for a single concept.¹⁰

The Operational and Financial Impact of Documentation Quality

Documentation quality is a direct driver of operational efficiency.⁷ Research indicates that developers spend approximately 35% of their working hours maintaining code and documentation rather than writing new features.⁷

Furthermore, users rarely read entire pages of technical content; studies show that readers consume only about 20% to 28% of the text on a webpage, with cognitive load increasing exponentially with every additional 100 words.⁹

To counter this, content must be structured for high scannability.²⁶ The UK Government, for instance, recommends limiting paragraphs to a maximum of five sentences, organized around a single, complete idea.⁹

When documentation fails to provide clear, scannable, and accessible information, organizations face direct costs:

- **Increased Support Overhead:** Unclear or misleading instructions force users to open support tickets, driving up customer service costs.⁴
- **Reduced Adoption Rates:** Poor developer experiences with APIs or technical

integrations can lead to project delays or abandonment in favor of competitor platforms with clearer documentation.⁷

- **Documentation Debt:** Allowing inconsistent, overly complex writing to accumulate in repositories creates "documentation debt" that requires significant engineering time to resolve.⁷

Programmatic Enforcement: The Docs-as-Code Workflow

Relying on manual editorial reviews to enforce inclusive language and eliminate diminishing terms is often slow and prone to human oversight.⁶ To address this, modern technical writing teams treat documentation with the same engineering rigor as software development.⁶ This "docs-as-code" methodology integrates automated prose linters directly into the content creation lifecycle.⁶

Prose linters analyze Markdown, HTML, or plain text against custom-defined style rules, flagging non-compliant terminology, passive voice, and condescending adverbs before the content is published.⁶

Vale: Syntax-Aware Prose Linting

Vale is an open-source, syntax-aware prose linter designed to enforce organizational style guides at scale.⁶ Vale's primary strength is its extensibility; teams can import pre-built packages based on established industry standards, such as the Google Developer Style Guide or the Microsoft Writing Style Guide, and merge them with custom, brand-specific vocabularies.⁶ Because Vale runs as a command-line interface (CLI) tool, it can be integrated into local text editors, Git pre-commit hooks, and continuous integration (CI/CD) pipelines.⁶ In a typical CI/CD workflow, any pull request containing documentation modifications is checked by Vale; if a style violation—such as the use of a non-inclusive term or a diminishing adverb—is detected, the build fails and the merge is blocked.⁷ This ensures consistent compliance without requiring manual oversight.⁷

Alex.js: Inclusive Writing Analysis

Alex.js is an open-source linter designed specifically to detect gender-favoring, polarizing, race-related, or ableist phrasing.²² Alex.js supports Markdown, HTML, and plain text, analyzing files and returning clear, actionable suggestions.²²

Importantly, Alex.js includes rules specifically targeting condescending language.²² It flags terms like *obviously*, *clearly*, or *everyone knows*, prompting the author to use more neutral, objective phrasing.²²

The technical differences and integration options for these automated tools are outlined in the table below:

Prose Linter	Primary Technology & Licensing	Core Capabilities	CI/CD & IDE Integration Methods	Target Rule Sets
Vale	Go-based; Open-source ⁶	Syntax-aware prose analysis; enforces style, capitalization, spelling, and grammar. ⁶	Local CLI; plugins for VS Code, Vim, Cursor; runs in GitHub Actions to block non-compliant PRs. ⁶	Google Style, Microsoft Style, custom enterprise vocabularies. ⁶
Alex.js	Node.js-based; MIT License ²²	Analyzes text for gender bias, ableist terms, racism, profanities, and condescending language. ²²	Integration via npm, Gulp, Slack bots, and standard text editor extensions. ³⁷	Built-in rules for inclusive, non-biased, and considerate writing. ²²
Fern	Proprietary with open components ⁷	Automates technical documentation quality checks at scale; incorporates link validation and spelling. ⁷	Native integration within specialized API documentation and publishing pipelines. ⁷	Developer experience guidelines, terminology standards, and API reference style. ⁷

Synthesized Conclusions and Strategic Recommendations

A comprehensive documentation strategy must look beyond high-level technical nomenclature to address the broader linguistic, structural, and cognitive barriers that impact usability.⁴ To build a modern, accessible, and scalable technical documentation ecosystem, organizations can implement several practical measures:

- Establish Open-Licensed Guidelines:** Adopt a highly regarded, open-licensed framework—such as the Google Developer Documentation Style Guide (CC-BY 3.0)—as the primary baseline.¹² This provides a pre-built foundation for handling complex grammar, global formatting, and inclusive language, allowing teams to focus on custom,

product-specific terms.¹¹

- **Establish a Policy Against Diminishing Adverbs:** Formally restrict the use of words that assume immediate comprehension or ease, such as *simply*, *just*, *obviously*, and *clearly*.¹⁷ Instruct writers to replace these terms with direct, imperative verbs (for example, using "configure the file" instead of "simply configure the file") to reduce cognitive load and support readers with neurodivergent conditions.⁹
- **Implement Cognitive Accessibility and Plain Language Standards:** Organize documentation around WCAG 2.2 Understandable principles.⁴ Restrict paragraphs to a maximum of five sentences, maintain clean heading hierarchies, omit confusing slash notations like "and/or," and provide clear visual aids to help readers process complex concepts.⁹
- **Shift to Asset-Based and Empowering Language:** Replace deficit-focused phrasing (such as "suffers from") with objective, neutral statements.¹⁵ Avoid patronizing terminology, and prioritize language that focuses on user capability and agency.²⁴
- **Automate Enforcement in CI/CD Pipelines:** Deploy open-source prose linters like Vale and Alex.js directly within Git workflows.⁶ By treating documentation quality with the same engineering rigor as software code, organizations can ensure that non-inclusive language, grammar errors, and diminishing adverbs are caught and resolved automatically before reaching production.⁶

Works cited

1. Tech Writing Style Guide: How to Build One - Heretto, accessed June 3, 2026, <https://www.heretto.com/blog/creating-a-technical-content-style-guide>
2. Style Guides - Write the Docs, accessed June 3, 2026, <https://www.writethedocs.org/guide/writing/style-guides/>
3. Inclusive Language Guide - Academy Software Foundation (ASWF), accessed June 3, 2026, <https://www.aswf.io/inclusive-language-guide/>
4. How to Write Accessible Content: Plain Language, Readability, and WCAG - Siteimprove, accessed June 3, 2026, <https://www.siteimprove.com/blog/readability-plain-language-wcag/>
5. Edit for accessibility | Technical Writing - Google for Developers, accessed June 3, 2026, <https://developers.google.com/tech-writing/accessibility/self-study/editing-accessibility>
6. Write Better with Vale: Automate Your Style Guides and Lint Prose Like You Lint Code by Brian P. Hogan, accessed June 3, 2026, <https://pragprog.com/titles/bhvale/write-better-with-vale/>
7. Docs Linting Guide - January 2026 | Fern, accessed June 3, 2026, <https://buildwithfern.com/post/docs-linting-guide>
8. Writing in Plain Language | accessibility.umich.edu, accessed June 3, 2026, <https://accessibility.umich.edu/how-to/web-content/sites-apps/writing-in-plain-language>

9. Inclusive Content Design: Addressing Cognitive and Neurodiverse Needs - Netguru, accessed June 3, 2026, <https://www.netguru.com/blog/inclusive-content-design-neurodiversity>
10. Writing tips - Microsoft Style Guide, accessed June 3, 2026, <https://learn.microsoft.com/en-us/style-guide/global-communications/writing-tips>
11. Technical writing resources - Google for Developers, accessed June 3, 2026, <https://developers.google.com/tech-writing/resources>
12. About this guide | Google developer documentation style guide, accessed June 3, 2026, <https://developers.google.com/style>
13. Documentation Guidelines - Creative Commons Open Source, accessed June 3, 2026, <https://opensource.creativecommons.org/contributing-code/documentation-guidelines/>
14. Google Style Guides | styleguide, accessed June 3, 2026, <https://google.github.io/styleguide/>
15. Write inclusive documentation | Google developer documentation style guide, accessed June 3, 2026, <https://developers.google.com/style/inclusive-documentation>
16. Contributing to PowerShell documentation - Microsoft Learn, accessed June 3, 2026, <https://learn.microsoft.com/en-us/powershell/scripting/community/contributing/overview?view=powershell-7.6>
17. Use simple words, concise sentences - Microsoft Style Guide, accessed June 3, 2026, <https://learn.microsoft.com/en-us/style-guide/word-choice/use-simple-words-concise-sentences>
18. Accessible - Microsoft Style Guide, accessed June 3, 2026, <https://learn.microsoft.com/en-us/style-guide/a-z-word-list-term-collections/a/accessible>
19. Writing style guide - MDN Web Docs, accessed June 3, 2026, https://developer.mozilla.org/en-US/docs/MDN/Writing_guidelines/Writing_style_guide
20. Plain Language and Accessibility | How to write accessibly - University of Arkansas Division of Agriculture, accessed June 3, 2026, <https://www.uada.edu/employees/division-accessibility/documents/plain-language.aspx>
21. Writing for Accessibility - Mailchimp Content Style Guide, accessed June 3, 2026, <https://styleguide.mailchimp.com/writing-for-accessibility/>
22. get-alex/alex: Catch insensitive, inconsiderate writing - GitHub, accessed June 3, 2026, <https://github.com/get-alex/alex>
23. Adding Text Linting to Eleventy - Raymond Camden, accessed June 3, 2026, <https://www.raymondcamden.com/2020/02/10/adding-text-linting-to-eleventy>
24. Using Plain Language to Support Individuals with Cognitive Disabilities | University of Colorado, accessed June 3, 2026,

- <https://www.cu.edu/coleman/resources/plain-language/using-plain-language-support-individuals-cognitive-disabilities>
25. Accessibility - How plain language improves access and usability, accessed June 3, 2026,
<https://service.tamu.edu/TDCClient/36/Portal/KB/Article/835/Accessibility-How-plain-language-improves-access-and-usability>
 26. Neurodivergence and Content Design: The Migraine Edition - DK Consulting of Colorado, accessed June 3, 2026,
<https://dkconsultingcolorado.com/2024/01/30/neurodivergence-and-content-design-the-migraine-edition/>
 27. Standards and Guidelines on digital accessibility - Australian Human Rights Commission, accessed June 3, 2026,
<https://humanrights.gov.au/resource-hub/by-resource-type/guidelines-and-standards/guides-and-standards-disability-rights/chapter-3-standards-and-guidelines-digital-accessibility>
 28. Cognitive and Learning Disabilities Accessibility Task Force Teleconference – 9 feb 2026, accessed June 3, 2026,
<https://www.w3.org/2026/02/09-coga-minutes.html>
 29. Inclusive Language Guide | Brand guidelines - University of Illinois Chicago, accessed June 3, 2026,
<https://brand.uic.edu/messaging/inclusive-language-guide/>
 30. INCLUSIVE WRITING GUIDE | Portland.gov, accessed June 3, 2026,
<https://www.portland.gov/sites/default/files/2022/inclusive-writing-guide-11-15-22.pdf>
 31. Technical Writing Guidelines, Style Guides, or Editorial Guides in Technical Writing - Quanos, accessed June 3, 2026,
<https://quanos.com/en/blog/technical-documentation/detail/guidelines-style-guide-editorial-guide/>
 32. Technical Writing Best Practices for Better Results, accessed June 3, 2026,
<https://www.hurleywrite.com/technical-writing-best-practices-guide/>
 33. TL;DR | Mailchimp Content Style Guide, accessed June 3, 2026,
<https://styleguide.mailchimp.com/tldr/>
 34. Scannable content - Microsoft Style Guide, accessed June 3, 2026,
<https://learn.microsoft.com/en-us/style-guide/scannable-content/>
 35. Syntax linting in Buildkite's Docs with Vale and Alex, accessed June 3, 2026,
<https://buildkite.com/resources/blog/linting-the-buildkite-docs/>
 36. Elastic style guide for Vale | Elastic Docs, accessed June 3, 2026,
<https://www.elastic.co/docs/contribute-docs/vale-linter>
 37. alex - catch insensitive, inconsiderate writing, accessed June 3, 2026,
<https://alexjs.com/>
 38. alex Inclusive Writing Linter | Skills - LobeHub, accessed June 3, 2026,
<https://lobehub.com/skills/agentskillexchange-skills-alex-inclusive-writing-linter>
 39. Bethbot - Beth Dunn, accessed June 3, 2026, <https://www.bethdunn.com/bethbot>
 40. WRITING_STYLE.md - microsoft/agent-academy - GitHub, accessed June 3, 2026,

https://github.com/microsoft/agent-academy/blob/main/WRITING_STYLE.md